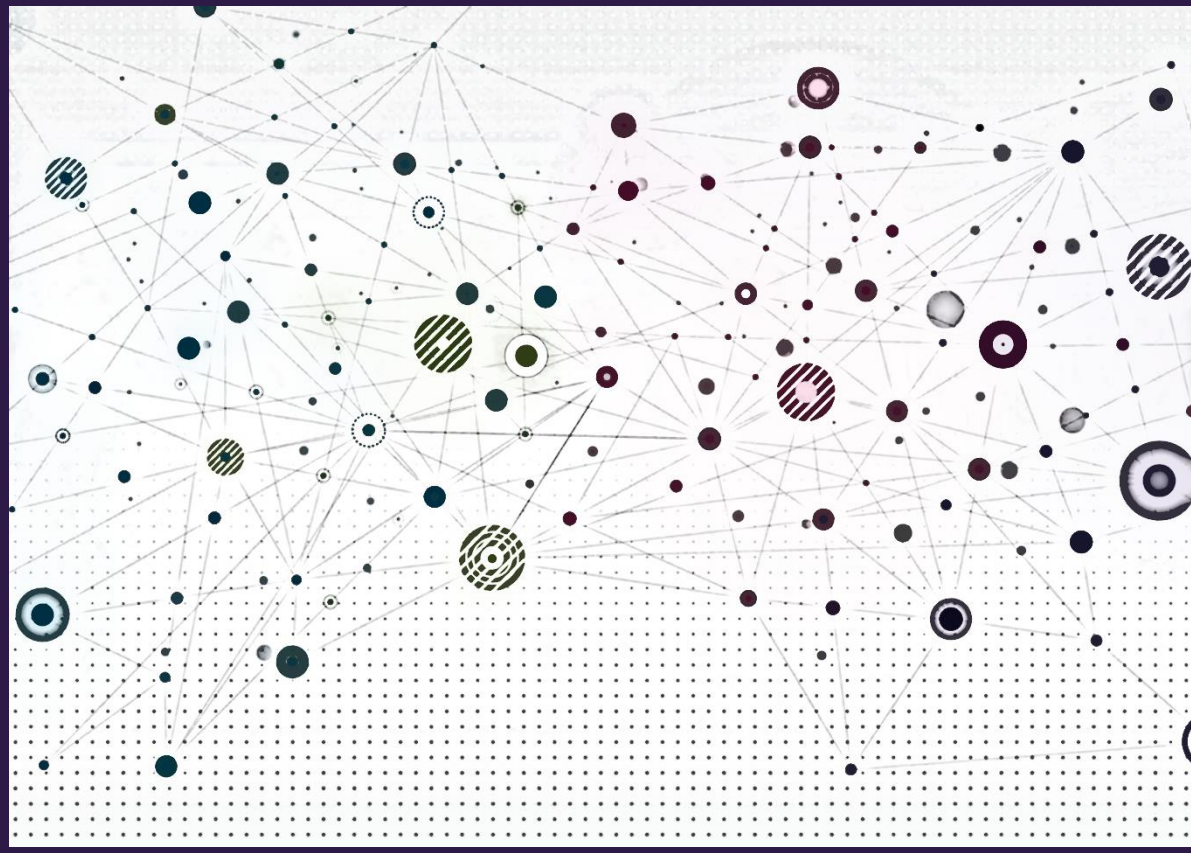
School of COMPUTING Clemson University

Heterogeneous Graphs



Overview

- Introduction to heterogeneous graphs
- Extending GCNs for heterogeneous graphs
- Inferences on relational graphs

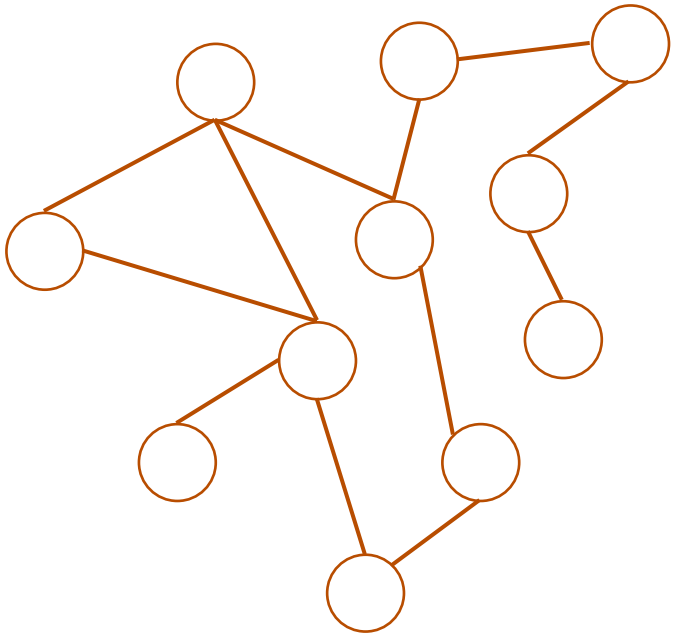
School of COMPUTING Clemson University

Heterogeneous graph introduction

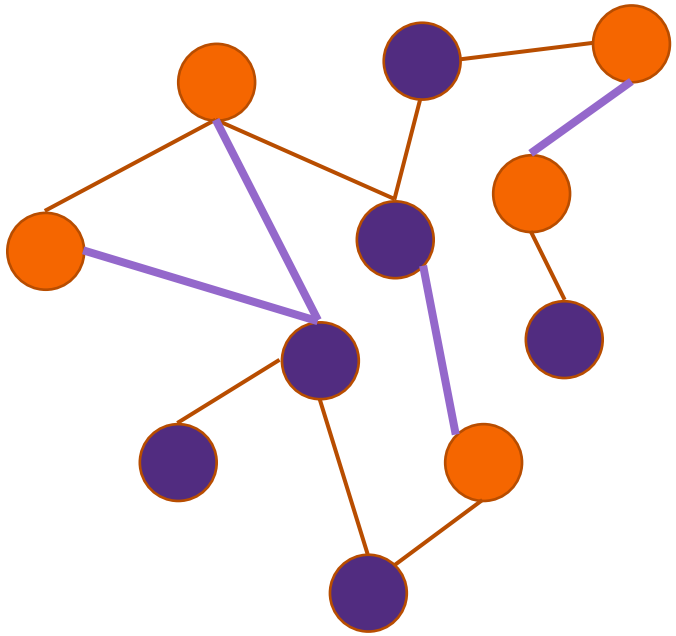
Heterogeneous vs. homogeneous graphs

To date, we have only considered homogeneous graphs

Homogeneous graphs– all nodes and edges are of the same type and have the same attributes



Heterogeneous graph– different node and/or edge types with possibly different attributes.



Heterogenous graphs example

Citation network with 2 types of nodes:

- Node type A: paper nodes
- Node type B: author nodes

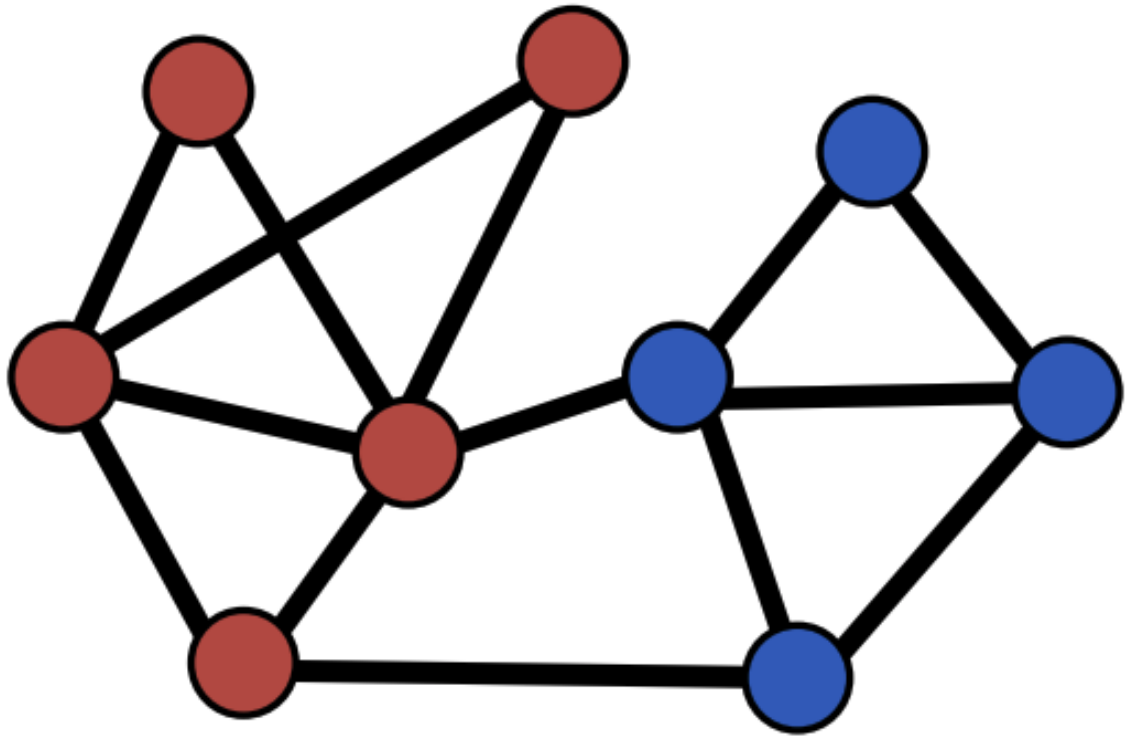


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Heterogenous graphs example

Citation network with 2 types of edges:

- Edge type A: cites
- Node type B: likes

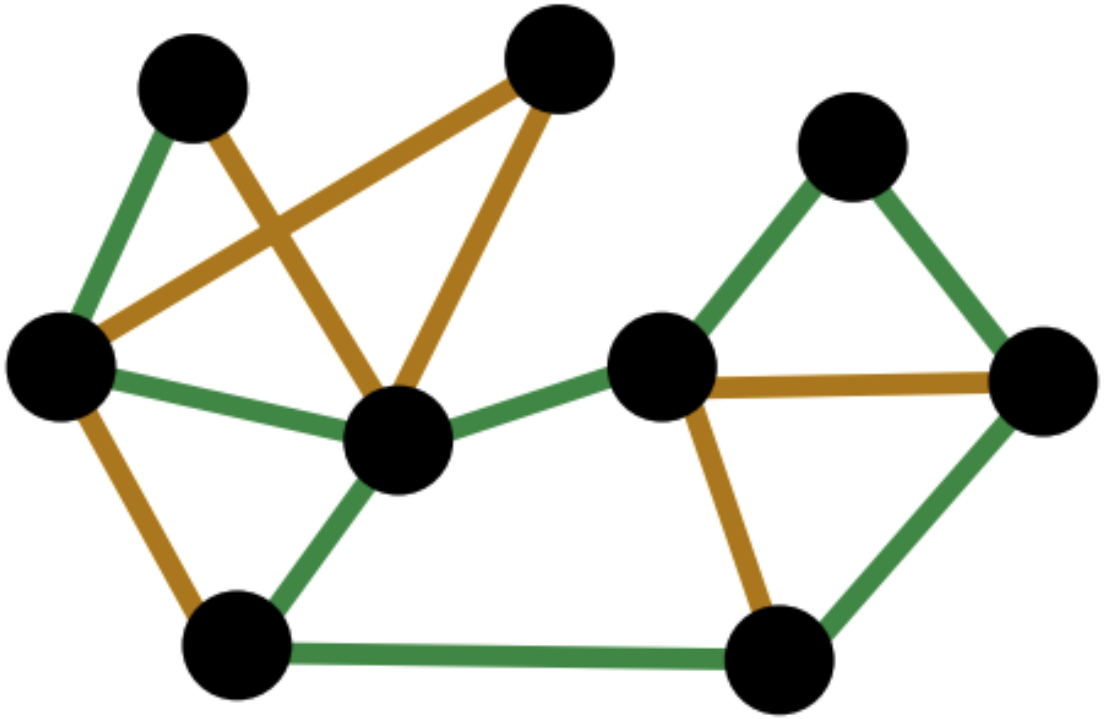


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Heterogenous graphs example

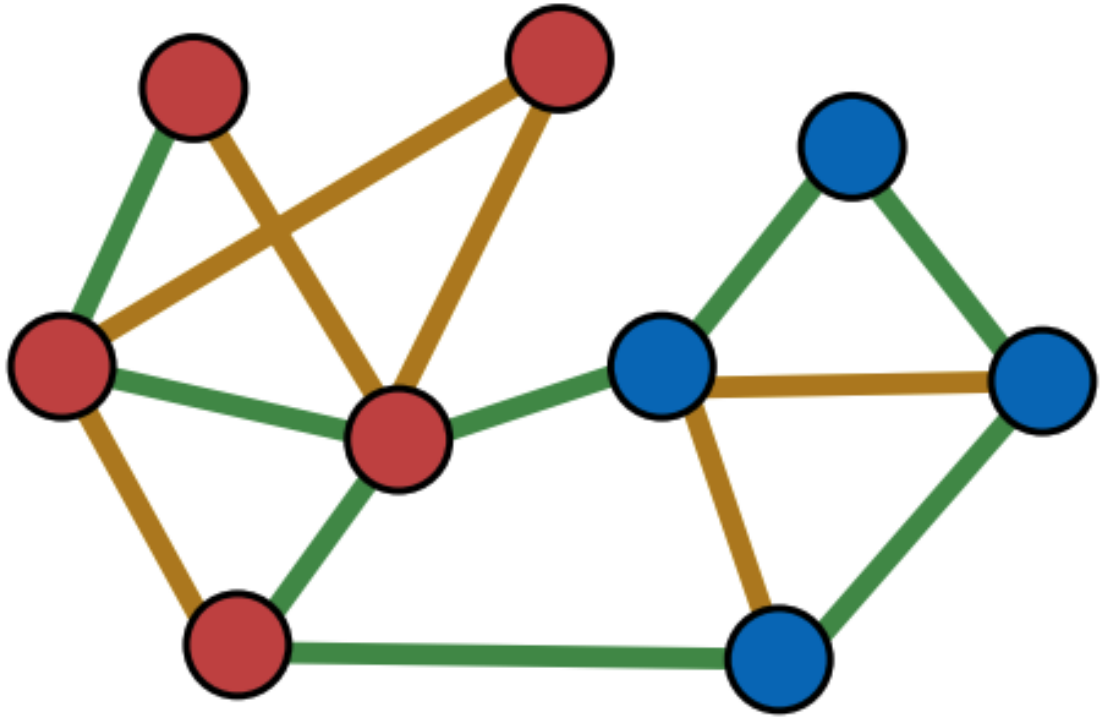
- Overall graph has 2 node and 2 edge types
- 8 possible **relation** types in this graph where the relation type is defined by the triple: (node_start type, edge type, node_end type)

(paper, cites, paper)

(paper, likes, paper)

(paper, cites, author)

(paper, likes, author)



(author, cites, author)

(author, likes, author)

(author, cites, paper)

(author, likes, author)

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Heterogeneous graphs terminology

- Define a heterogeneous graph as

$$G = (\boldsymbol{V}, \boldsymbol{E}, \tau, \phi)$$

where

- Nodes $v \in \boldsymbol{V}$ have a node type $\tau(v)$
  - Example: $\tau(v) \in \{author, paper\}$
- Edges $(u, v) \in \boldsymbol{E}$ have an edge type $\phi(u, v)$
  - Example: $\phi(u, v) \in \{likes, cites\}$
- Relation types $r = \big(\tau(u), \phi(u, v), \tau(v)\big)$
  - Example: (author, cites, paper)
- Different node and edge types can have different feature spaces

Many graphs are heterogeneous

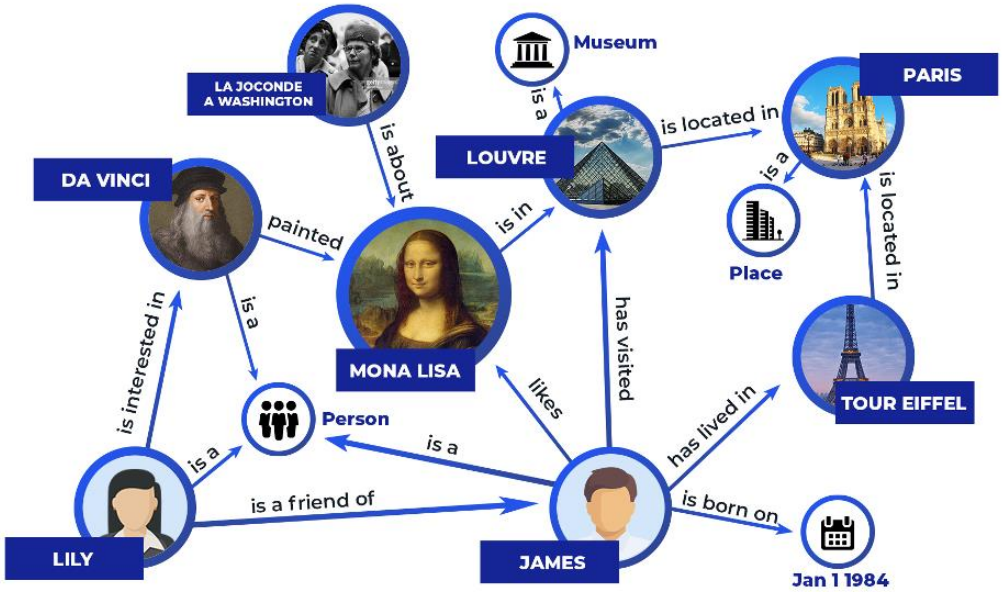


Image credit: Zilliz

Knowledge Graphs

Node types: Artifact, Museum, Person, Place

Edge types: is_a, is_a_friend_of, is_born_on

Example Relation: Lilly, is_friend_of, James

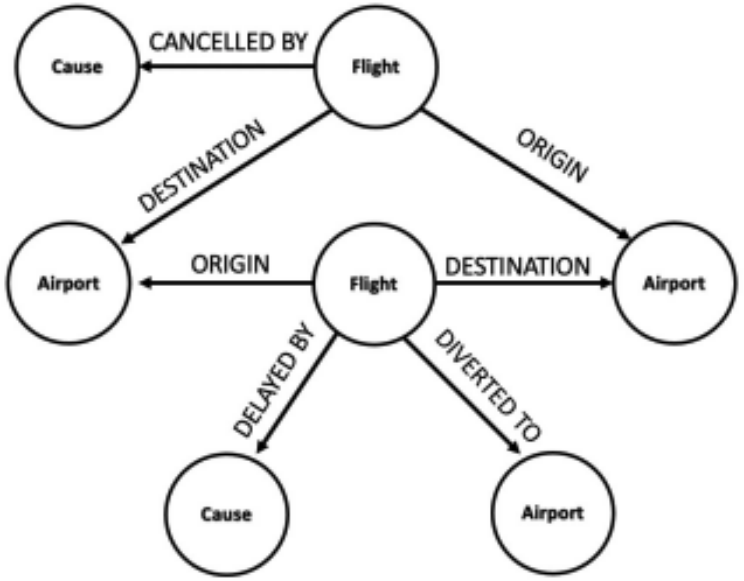


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Event Graphs

Node types: Airport, Cause, Flight

Edge types: Origin, Delayed By, Cancelled By

Example Relation: Flight, Delayed BY, Cause

Many graphs are heterogeneous

E-commerce graph

- Node types: User, Item, Query, …
- Edge types: Purchase, Visit, Guide, Search, …
- Different node types have different feature spaces

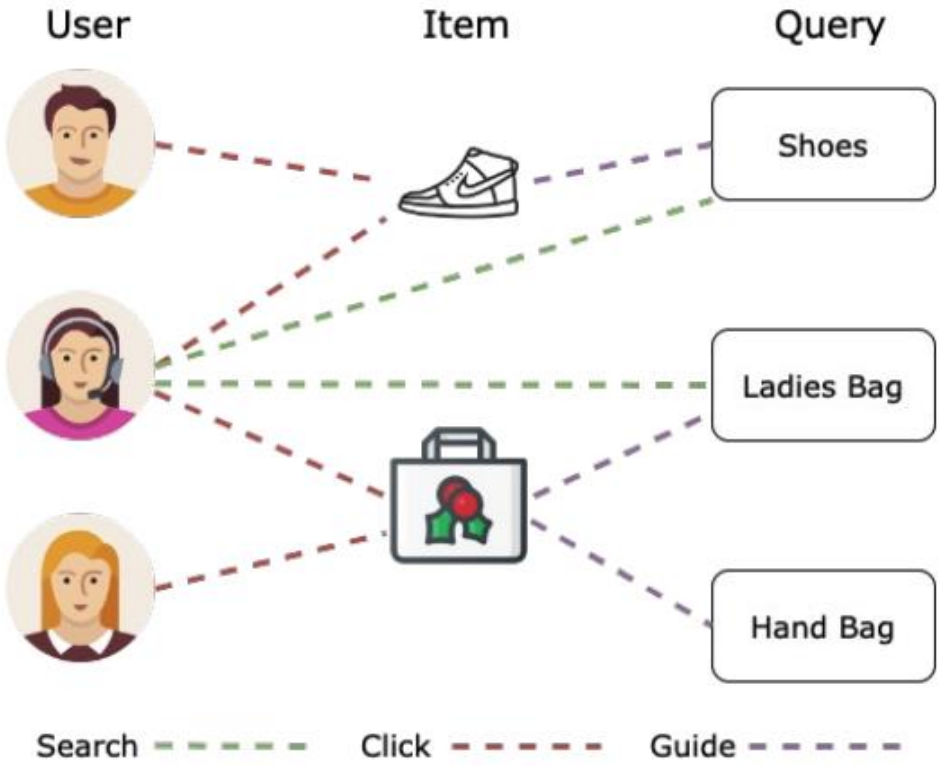


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Many graphs are heterogeneous

Microsoft Open Academic Graph

- Node types: Author, Paper, Venue, Field, …
- Edge types: Publish, Cite, Belong, …
- >300M papers, >64M edges
- https://www.microsoft.com/en-us/research/project/open-academic-graph/

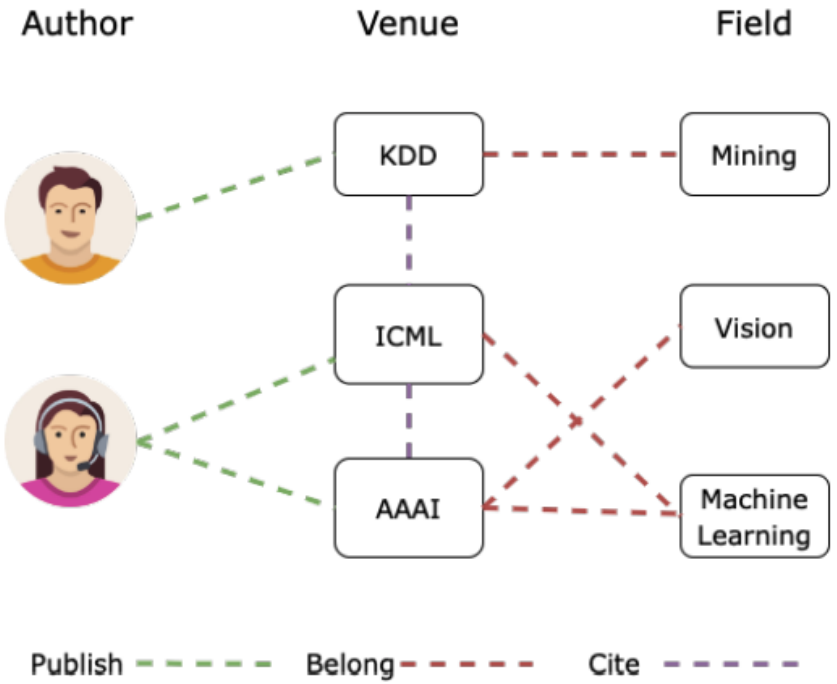


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Types as features?

- We can potentially treat node and edge types as features
  - Append a one-hot encoding to the node or edge feature vector
    - Append $[1,0]$ to the feature vector of author nodes and $[0,1]$ for paper nodes
    - Append $[1,0,0]$ to "cites" edges, $[0,1,0]$ to "publish", and $[0,0,1]$ to "belong"
- Heterogeneous graph reduces to a homogeneous graph in this case
- When do we need a heterogeneous graph?

When do we need a heterogeneous graph?

- Case 1: Different node / edge types have different feature dimensions
  - Example: ”Author” nodes have 4 features while ”Paper” nodes have 10 features
- Case 2: Different relation types (n,e,n) represent different interaction for which we expect different models are needed
  - Example: (English, translate, French) requires a different model than (English, translate, Mandarin Chinese)
- The tradeoff: a heterogeneous graph is a more expressive representation, but it is also has
  - increased computation and storage costs relative to homogeneous graphs
  - more complex implementation

Representing heterogeneous graphs in PyG

- HeteroData class supports creation of heterogeneous graph instances
- https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.data.HeteroData.html#torch_geometric.data.HeteroData

HeteroData

class **HeteroData** (**_mapping**: Optional[Dict[str, Any]] = **None**, ****kwargs**) [source]

Bases: `BaseData`, `FeatureStore`, `GraphStore`

A data object describing a heterogeneous graph, holding multiple node and/or edge types in disjunct storage objects. Storage objects can hold either node-level, link-level or graph-level attributes. In general, `HeteroData` tries to mimic the behavior of a regular **nested** Python dictionary. In addition, it provides useful functionality for analyzing graph structures, and provides basic PyTorch tensor functionalities.

```
from torch_geometric.data import HeteroData

data = HeteroData()

# Create two node types "paper" and "author" holding a feature matrix:
data['paper'].x = torch.randn(num_papers, num_paper_features)
data['author'].x = torch.randn(num_authors, num_authors_features)

# Create an edge type "(author, writes, paper)" and building the
# graph connectivity:
data['author', 'writes', 'paper'].edge_index = ...  # [2, num_edges]

data['paper'].num_nodes
>>> 23

data['author', 'writes', 'paper'].num_edges
>>> 52

# PyTorch tensor functionality:
data = data.pin_memory()
data = data.to('cuda:0', non_blocking=True)
```

School of COMPUTING Clemson University

Extending GCNs for heterogeneous graphs

Graph convolutional networks

- A GCN can be written as

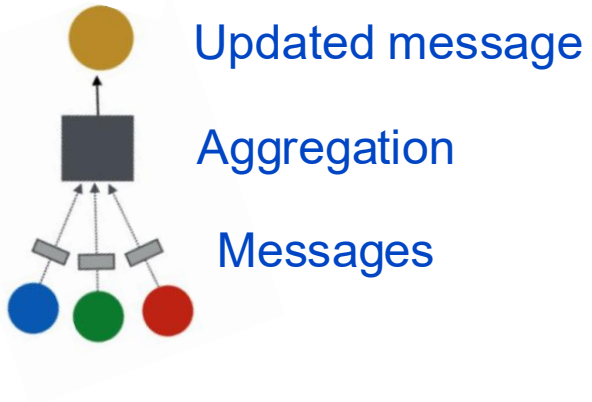
$$\mathbf{h}_v^{(k+1)} = \sigma\left(\mathbf{W}^{(k)} \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(k)}}{|N(v)|}\right)$$

NOTE: No self loop here, we'll address this later

- How to write this as aggregation → update?

$$\mathbf{m}_{N(v)}^{(k)} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(k)}}{|N(v)|}$$

$$\mathbf{h}_v^{(k+1)} = \sigma\left(\mathbf{W}^{(k)} \mathbf{m}_{N(v)}^{(k)}\right)$$



Relational graph convolutional networks (RGCN)

- Extend GCN to handle heterogeneous graphs with multiple relation types
- Start with a **directed** graph with **one** relation
  - How do we run GCN and update the representation of target node A on this graph?

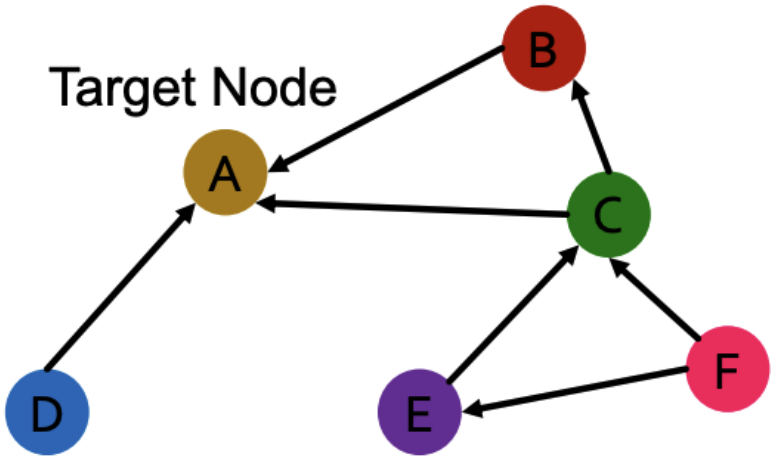
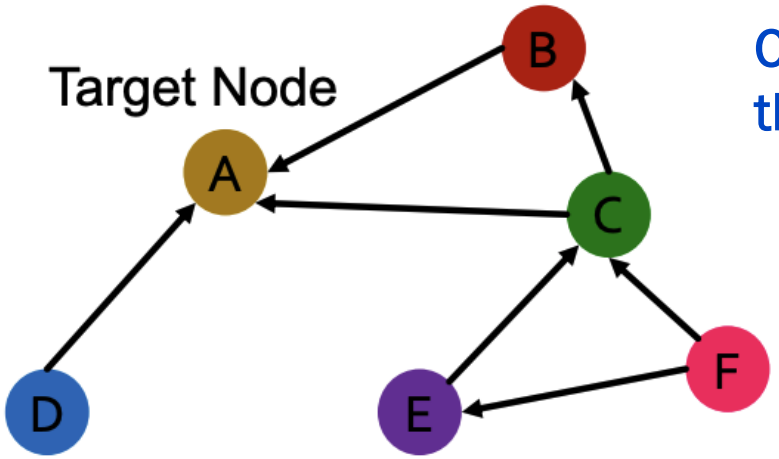


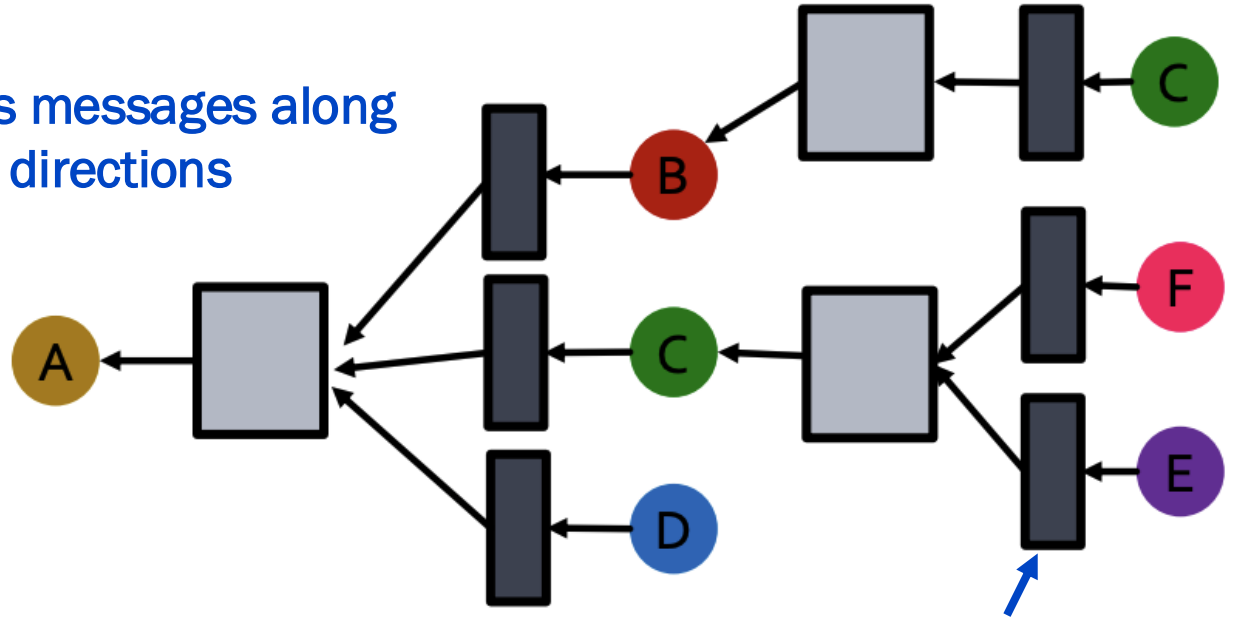
Image credit: Jure Leskovec, http://cs224w.stanford.edu

Relational graph convolutional networks (RGCN)

- Extend GCN to handle heterogeneous graphs with multiple relation types
- Start with a **directed** graph with **one** relation
  - How do we run GCN and update the representation of target node A on this graph?
  - This is implemented via the appropriate adjacency matrix (i.e., for the relation)



Only pass messages along the edge directions



Same neural network across all aggregations

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Relational graph convolutional networks (RGCN)

- What if the graph has multiple relation types?

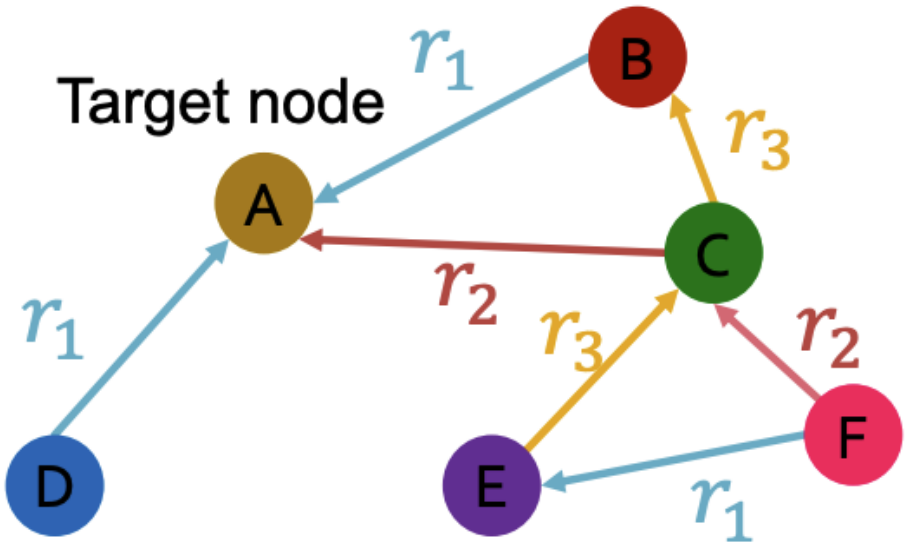
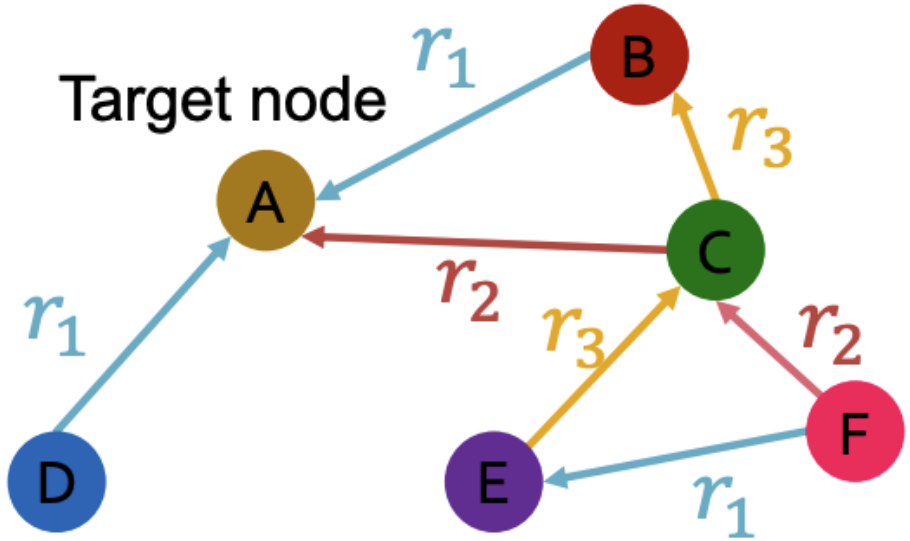


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Relational graph convolutional networks (RGCN)

- What if the graph has multiple relation types?
- Use different neural networks for different relation types



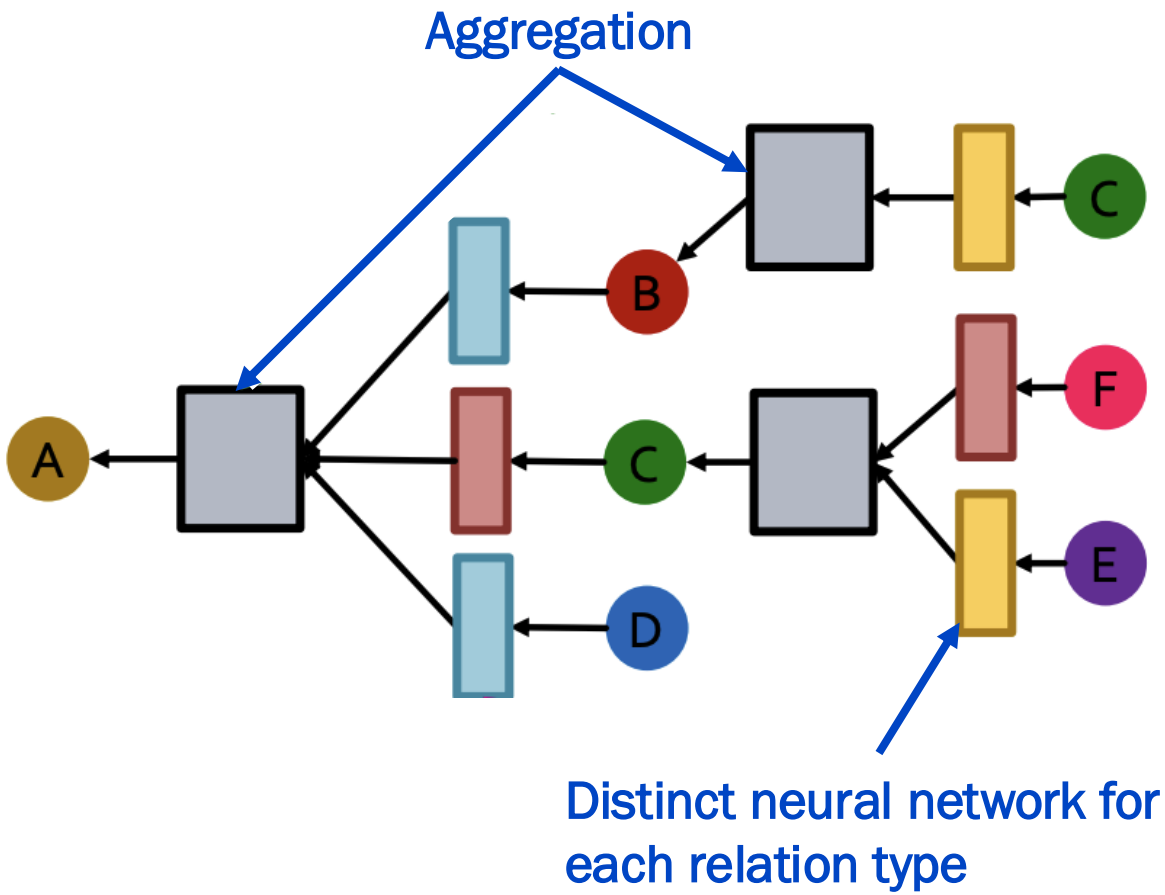


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Graph convolutional networks with self-loop

- A GCN can be written as

$$\mathbf{h}_v^{(k+1)} = \sigma\left(\mathbf{W}^{(k)} \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(k)}}{|N(v)|}\right)$$

- Let's add a self-loop with its own linear projection

$$\mathbf{h}_v^{(k+1)} = \sigma\left(\mathbf{W}^{(k)} \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(k)}}{|N(v)|} + \mathbf{W}^{(k)} \mathbf{h}_v^{(k)}\right)$$

Relational graph convolutional networks (RGCN)

- RGCN includes a neural network for each relation type
- In the node update, it sums the node embeddings for each relation

$$\mathbf{h}_v^{(k+1)} = \sigma\left(\left[\sum_{r\in R}\sum_{u\in N_r(v)}\frac{1}{c_{v,r}}\mathbf{W}_r^{(k)}\mathbf{h}_u^{(k)}\right] + \mathbf{W}_0^{(k)}\mathbf{h}_v^{(k)}\right)$$

where $N_r(v)$ is the neighborhood of node v for relation type r and $c_{v,r} = |N_r(v)|$

- In partial matrix form, the update for all nodes is given by

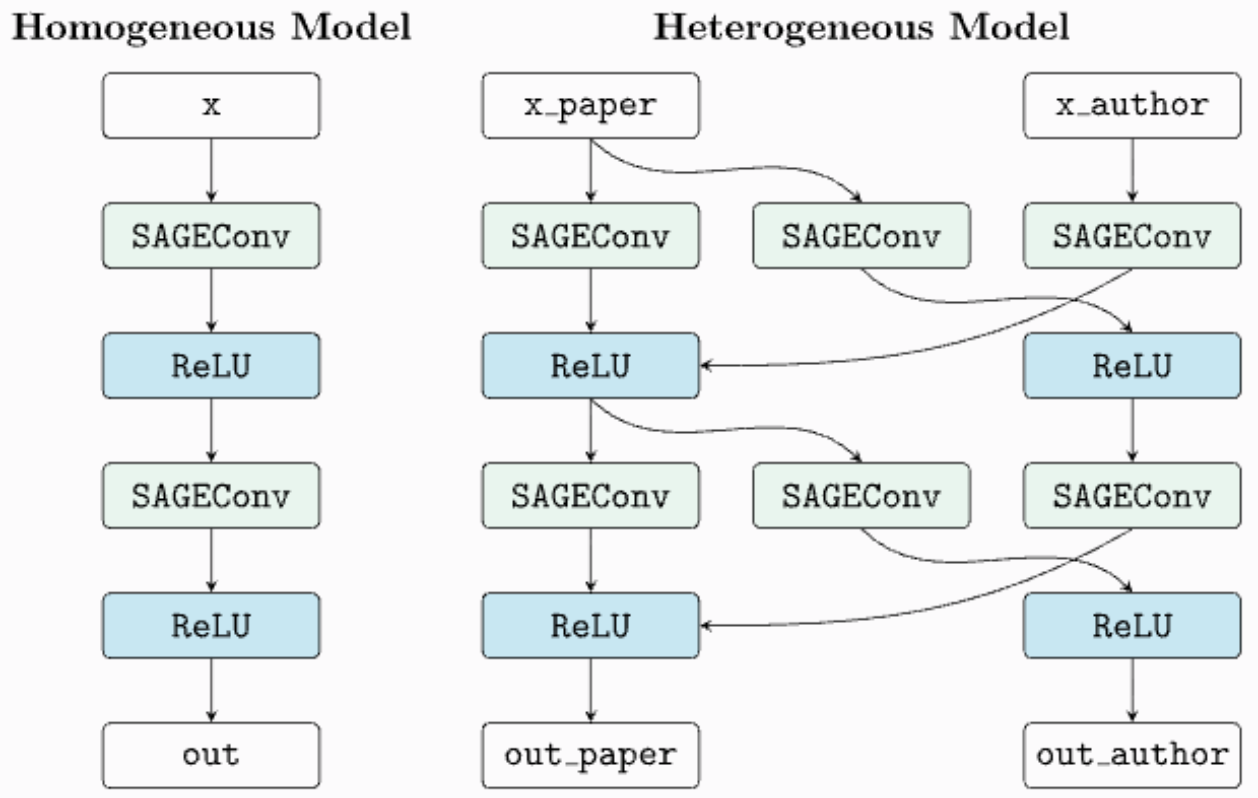
$$\mathbf{H}^{(k+1)} = \sigma\left(\left[\sum_{r\in R}\mathbf{D}_r^{-1}\mathbf{A}_r\mathbf{H}^{(k)}\mathbf{W}_r^{(k)}\right] + \mathbf{W}_0^{(k)}\mathbf{H}^{(k)}\right)$$

where $\mathbf{D}_r^{-1}$ is a diagonal matrix where the $(\mathbf{D}_r^{-1})_{v,v} = {}^{1}/_{|N_r(v)|}$

RGCN PyG implementation

- The PyG *to_hetero* method converts a homogeneous GNN model into its heterogeneous equivalent

```python
import torch
from torch_geometric.nn import SAGEConv, to_hetero

class GNN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = SAGEConv((-1, -1), 32)
        self.conv2 = SAGEConv((32, 32), 32)

    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index).relu()
        x = self.conv2(x, edge_index).relu()
        return x

model = GNN()

node_types = ['paper', 'author']
edge_types = [
    ('paper', 'cites', 'paper'),
    ('paper', 'written_by', 'author'),
    ('author', 'writes', 'paper'),
]
metadata = (node_types, edge_types)

model = to_hetero(model, metadata)
model(x_dict, edge_index_dict)
```

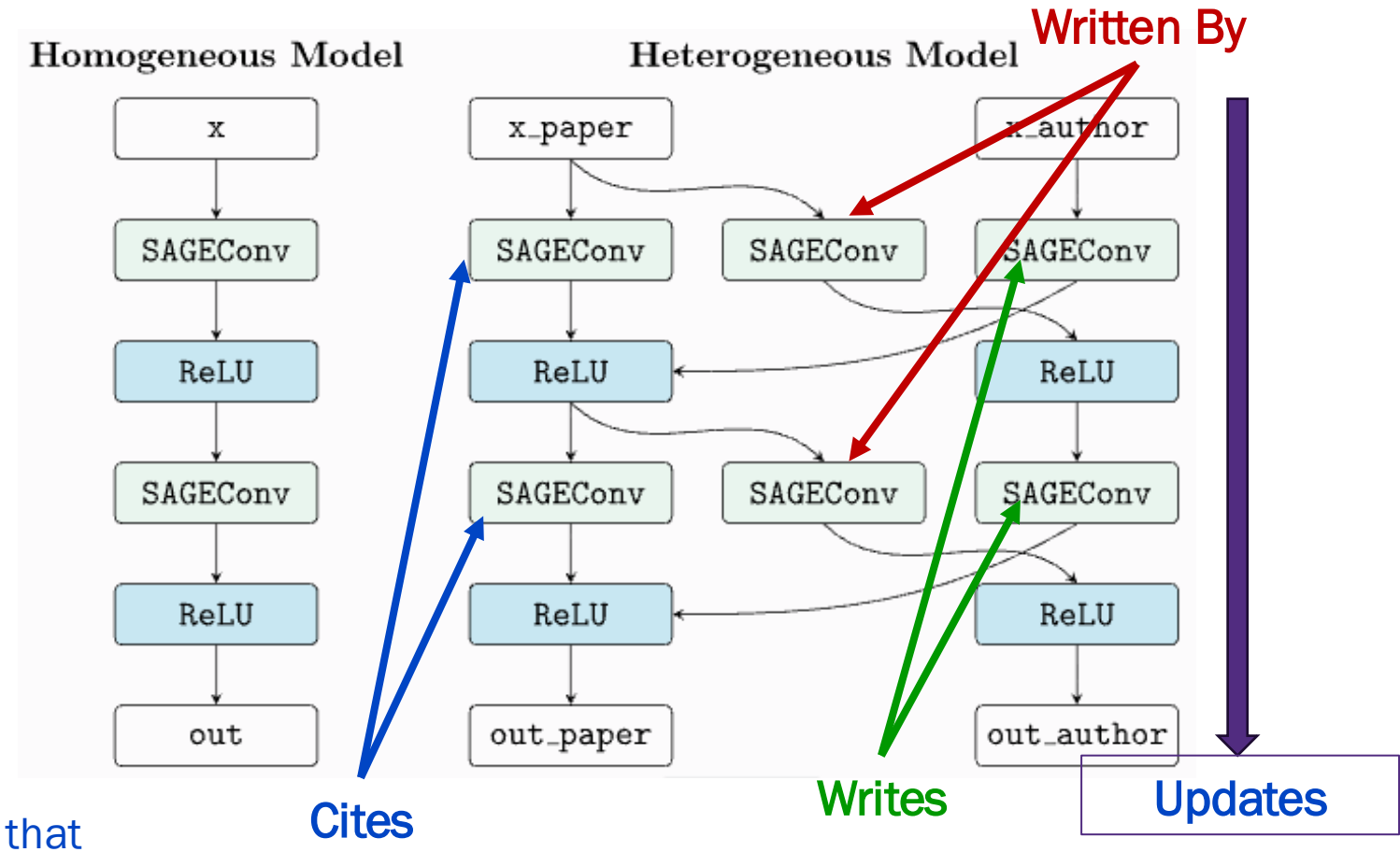


x_dict and *edge_index_dict* are dictionaries that hold node feature and edge connection information for each node and edge type, respectively

RGCN PyG implementation

- The PyG *to_hetero* method converts a homogeneous GNN model into its heterogeneous equivalent

```python
import torch
from torch_geometric.nn import SAGEConv, to_hetero

class GNN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = SAGEConv((-1, -1), 32)
        self.conv2 = SAGEConv((32, 32), 32)

    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index).relu()
        x = self.conv2(x, edge_index).relu()
        return x

model = GNN()

node_types = ['paper', 'author']
edge_types = [
    ('paper', 'cites', 'paper'),
    ('paper', 'written_by', 'author'),
    ('author', 'writes', 'paper'),
]
metadata = (node_types, edge_types)

model = to_hetero(model, metadata)
model(x_dict, edge_index_dict)
```



x_dict and *edge_index_dict* are dictionaries that hold node feature and edge connection information for each node and edge type, respectively

https://pytorch-geometric.readthedocs.io/en/latest/cheatsheet/gnn_cheatsheet.html

RGCN PyG implementation

- The PyG *to_hetero* method – not available for all GNN layers

```
import torch
from torch_geometric.nn import SAGEConv, to_hetero

class GNN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = SAGEConv((-1, -1), 32)
        self.conv2 = SAGEConv((32, 32), 32)

    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index).relu()
        x = self.conv2(x, edge_index).relu()
        return x

model = GNN()

node_types = ['paper', 'author']
edge_types = [
    ('paper', 'cites', 'paper'),
    ('paper', 'written_by', 'author'),
    ('author', 'writes', 'paper'),
]
metadata = (node_types, edge_types)

model = to_hetero(model, metadata)
model(x_dict, edge_index_dict)
```

x_dict and *edge_index_dict* are dictionaries that hold node feature and edge connection information for each node and edge type, respectively

Graph Neural Network Operators

Name	SparseTensor	edge_weight	edge_attr	bipartite	static	lazy
SimpleConv	✓	✓		✓	✓	
GCNConv (Paper)	✓	✓			✓	✓
ChebConv (Paper)		✓			✓	✓
SAGEConv (Paper)	✓			✓	✓	✓
CuGraphSAGEConv (Paper)					✓	
GraphConv (Paper)	✓	✓		✓	✓	✓
GatedGraphConv (Paper)	✓	✓			✓	
ResGatedGraphConv (Paper)	✓		✓	✓	✓	✓
GATConv (Paper)	✓		✓	✓		✓
CuGraphGATConv (Paper)			✓		✓	
FusedGATConv (Paper)					✓	
GATv2Conv (Paper)	✓		✓	✓		✓
TransformerConv (Paper)	✓		✓	✓		✓
AGNNConv (Paper)	✓				✓	
TAGConv (Paper)	✓	✓			✓	✓
GINConv (Paper)	✓			✓	✓	

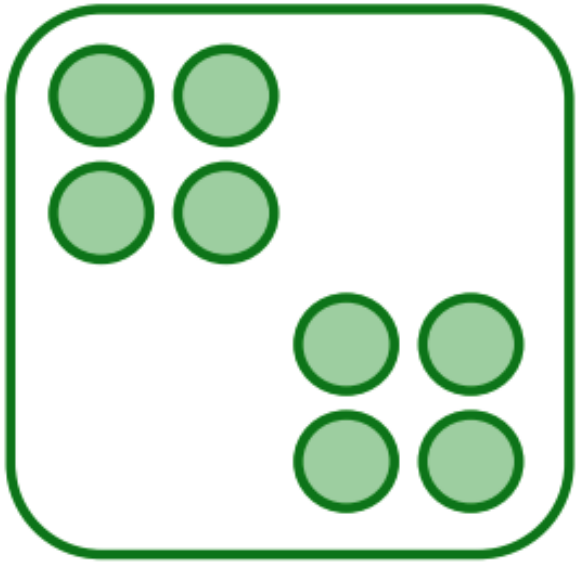
RGCN parameter growth

- Each relation, type r, has L matrices (1 per GNN layer): $\mathbf{W}_r^{(1)}$, $\mathbf{W}_r^{(2)}$, …, $\mathbf{W}_r^{(L)}$
- The size of each $\mathbf{W}_r^{(k)}$ is $d^{k+1} \times d^k$
- **Rapid growth in the number of parameters w.r.t. the number of relations**
  - Model complexity and overfitting are concerns
- Consider two methods to regularize the weights, $\mathbf{W}_r^{(k)}$
  - Block diagonal matrices
  - Basis / dictionary learning

RGCN with block diagonal weight matrices

- Goal is to make the weight matrices sparse
- Make each $\mathbf{W}_r^{(k)}$ a **block diagonal matrix**
- If we use B blocks, the number of learning parameters reduces from $d^{(k+1)} \times d^{(k)}$ to $B \times \frac{d^{(k+1)}}{B} \times \frac{d^{(k)}}{B}$

$$\mathbf{W}_r^{(k)} =$$



Limitation: only nearby features can interact through $\mathbf{W}_r^{(k)}$

RGCN with block diagonal weight matrices

- Make each $\mathbf{W}_r^{(k)}$ a **block diagonal matrix**
- If we use B blocks, the number of learning parameters reduces from $d^{(k+1)} \times d^{(k)}$ to $B \times \frac{d^{(k+1)}}{B} \times \frac{d^{(k)}}{B}$
- Example
  - $\mathbf{W}_r$ with $d^{(k+1)} = d^{(k)} = 6$ and $B = 3$
  - Let's consider $\tilde{\mathbf{h}}_u = \mathbf{W}_r \mathbf{h}_u \in \mathbb{R}^{6\times 1}$

Every entry (feature) in $\tilde{\mathbf{h}}_u$ is influenced by **every** feature in $\mathbf{h}_u$

$$\mathbf{W}_r^{(k)} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} & w_{15} & w_{16} \\ w_{21} & w_{22} & w_{23} & w_{24} & w_{25} & w_{26} \\ w_{31} & w_{32} & w_{33} & w_{34} & w_{35} & w_{36} \\ w_{41} & w_{42} & w_{43} & w_{44} & w_{45} & w_{46} \\ w_{51} & w_{52} & w_{53} & w_{54} & w_{55} & w_{56} \\ w_{61} & w_{62} & w_{63} & w_{64} & w_{65} & w_{66} \end{bmatrix} \quad \tilde{\mathbf{h}}_u = \begin{bmatrix} w_{11}\mathbf{h}_{u,1} + w_{12}\mathbf{h}_{u,2} + w_{13}\mathbf{h}_{u,3} + w_{14}\mathbf{h}_{u,4} + w_{15}\mathbf{h}_{u,5} + w_{16}\mathbf{h}_{u,6} \\ \vdots \\ w_{61}\mathbf{h}_{u,1} + w_{62}\mathbf{h}_{u,2} + w_{63\ 13}\mathbf{h}_{u,3} + w_{64}\ w_{14}\mathbf{h}_{u,4}\ w_{65}\mathbf{h}_{u,5} + w_{66}\mathbf{h}_{u,6} \end{bmatrix}$$

Standard approach requires $d^{(k+1)} \times d^{(k)} = 36$ learning parameters

RGCN with block diagonal weight matrices

- Make each $\mathbf{W}_r^{(k)}$ a **block diagonal matrix**
- If we use B blocks, the number of learning parameters reduces from $d^{(k+1)} \times d^{(k)}$ to $B \times \frac{d^{(k+1)}}{B} \times \frac{d^{(k)}}{B}$
- Example
  - $\mathbf{W}_r$ with $d^{(k+1)} = d^{(k)} = 6$ and $B = 3$
  - Let's consider $\tilde{\mathbf{h}}_u = \mathbf{W}_r \mathbf{h}_u \in \mathbb{R}^{6\times 1}$

Each feature is a transformation of a subset of the overall features (similar to a factor analysis)

$$\mathbf{W}_r^{(k)} = \begin{bmatrix} w_{11} & w_{12} & 0 & 0 & 0 & 0 \\ w_{21} & w_{22} & 0 & 0 & 0 & 0 \\ 0 & 0 & w_{33} & w_{34} & 0 & 0 \\ 0 & 0 & w_{43} & w_{44} & 0 & 0 \\ 0 & 0 & 0 & 0 & w_{55} & w_{56} \\ 0 & 0 & 0 & 0 & w_{65} & w_{66} \end{bmatrix} \quad \tilde{\mathbf{h}}_u = \begin{bmatrix} w_{11}\,\mathbf{h}_{u,1} + w_{12}\,\mathbf{h}_{u,2} \\ w_{21}\,\mathbf{h}_{u,1} + w_{12}\,\mathbf{h}_{u,2} \\ w_{33}\,\mathbf{h}_{u,3} + w_{34}\,\mathbf{h}_{u,4} \\ w_{43}\,\mathbf{h}_{u,3} + w_{44}\,\mathbf{h}_{u,4} \\ w_{55}\,\mathbf{h}_{u,5} + w_{56}\,\mathbf{h}_{u,5} \\ w_{65}\,\mathbf{h}_{u,5} + w_{66}\,\mathbf{h}_{u,6} \end{bmatrix}$$

Bloc approach requires $B \times \frac{d^{(k+1)}}{B} \times \frac{d^{(k)}}{B} = 12$ learning parameters

School of COMPUTING Clemson University

RGCN with block diagonal weight matrices

- Not currently implemented in PyG
- Implementation: do **not** create a sparse matrix
- Create the blocks and use batch matrix multiply
- In PyTorch / PyG the parameters would be created with nn.Parameter (see link below)

```
# Stack blocks into shape (B, block_size, block_size)
W_r_blocks = torch.stack([block_1, block_2, block_3])

# Reshape h_u from (6,) to (B, block_size)
h_u_reshaped = h_u.view(3, 2)  # Shape: (3, 2)

# Batched matrix-vector multiply
output = torch.bmm(W_r_blocks, h_u_reshaped.unsqueeze(-1)).squeeze(-1)

# Flatten back to (6,)
output = output.view(-1)
```

https://github.com/masino-teaching/ml4g/blob/main/notebooks/RGCN-with-block-diagonal-weights.ipynb

```
In [3]: Wr = torch.stack([torch.rand(2,2) for _ in range(3)])

In [4]: Wr.shape
Out[4]: torch.Size([3, 2, 2])

In [5]: h = torch.rand(6,1)

In [6]: h
Out[6]:
tensor([[0.5272],
        [0.1507],
        [0.4108],
        [0.2431],
        [0.0395],
        [0.2747]])

In [7]: h_reshaped = h.view(3,2)

In [8]: h_reshaped
Out[8]:
tensor([[0.5272, 0.1507],
        [0.4108, 0.2431],
        [0.0395, 0.2747]])

In [9]: h_update = torch.bmm(Wr, h_reshaped.unsqueeze(-1)).squeeze(-1)

In [10]: h_update
Out[10]:
tensor([[0.2952, 0.3379],
        [0.5286, 0.6133],
        [0.0317, 0.2690]])

In [11]: h_update = h_update.view(-1)

In [12]: h_update
Out[12]: tensor([0.2952, 0.3379, 0.5286, 0.6133, 0.0317, 0.2690])

In [13]: Wr
Out[13]:
tensor([[[0.3617, 0.6938],
         [0.5346, 0.3718]],

        [[0.8256, 0.7796],
         [0.9789, 0.8691]],

        [[0.2915, 0.0735],
         [0.9806, 0.8382]]])

In [14]: Wr[0].shape
Out[14]: torch.Size([2, 2])

In [15]: Wr[0,0,0]*h[0,0]+Wr[0,0,1]*h[1,0]
Out[15]: tensor(0.2952)
```

RCGN with basis learning

- Share weights across different relations
- Represent each relation matrix $\mathbf{W}_r^{(k)}$ as a linear combination of basis matrices

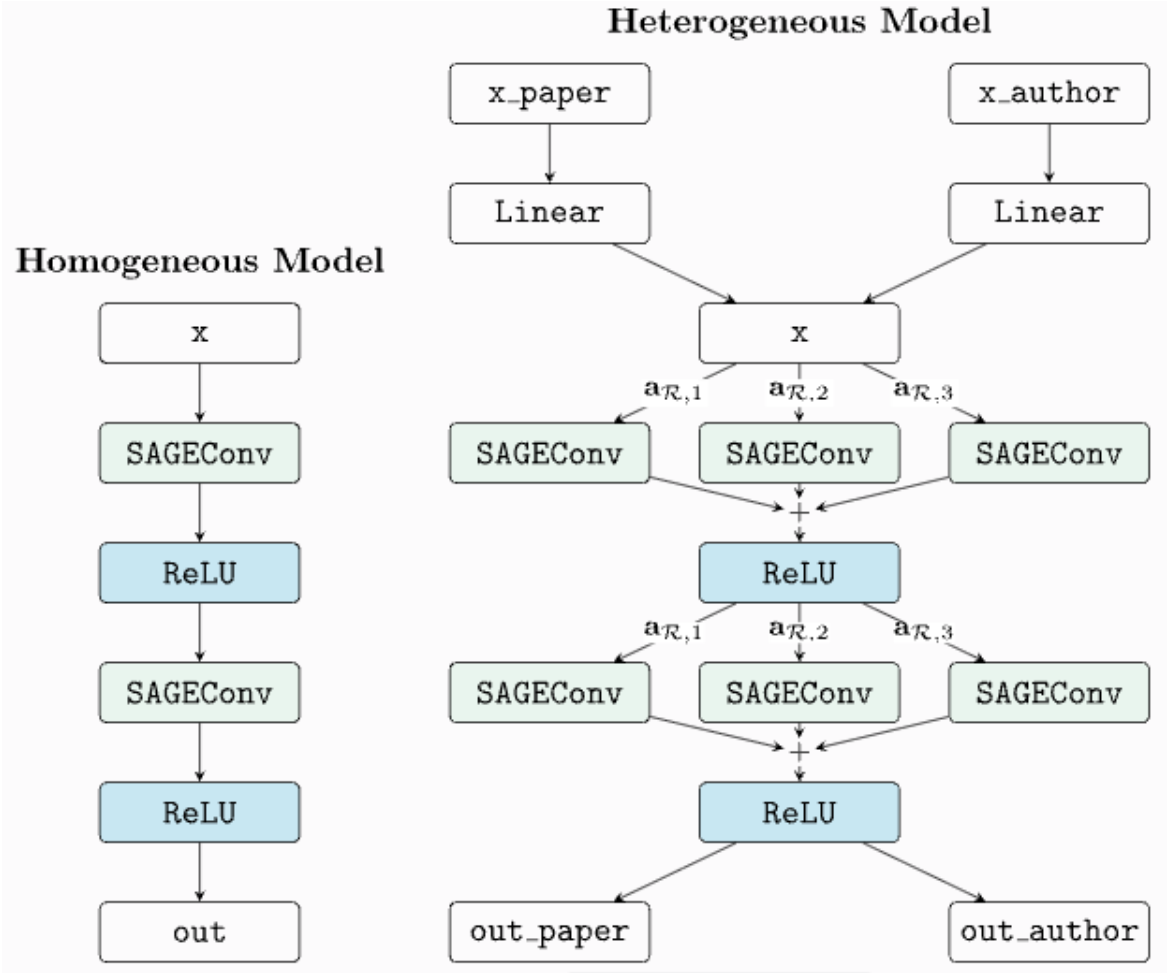
$$\mathbf{W}_r^{(k)} = \sum_{b=1}^{B} a_{r,b} \mathbf{V}_b^{(k)}$$

  where $\mathbf{V}_b^{(k)}$ are basis matrices and $a_{r,b}$ is a scalar weighting
- Now each relation only adds B learning parameters $\{a_{r,1}, \cdots, a_{r,B}\}$

RGCN PyG implementation

- The PyG *to hetero with bases* method converts a homogeneous GNN model into its heterogeneous equivalent using bases

```python
import torch
from torch_geometric.nn import SAGEConv, to_hetero_with_bases

class GNN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = SAGEConv((16, 16), 32)
        self.conv2 = SAGEConv((32, 32), 32)

    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index).relu()
        x = self.conv2(x, edge_index).relu()
        return x

model = GNN()

node_types = ['paper', 'author']
edge_types = [
    ('paper', 'cites', 'paper'),
    ('paper', 'written_by', 'author'),
    ('author', 'writes', 'paper'),
]
metadata = (node_types, edge_types)

model = to_hetero_with_bases(model, metadata, num_bases=3,
                             in_channels={'x': 16})
model(x_dict, edge_index_dict)
```



x_dict and *edge_index_dict* are dictionaries that hold node feature and edge connection information for each node and edge type, respectively

School of COMPUTING Clemson University

Inferences on relational graphs

Heterogeneous graph / node classification

- Goal: Infer the label of a given node
- Use the node embedding, $\mathbf{h}_v^{(K)} \in \mathbb{R}^{d_K}$ from the final RGCN layer, K
- Pass this embedding through the prediction head
  - Example: linear layer $\mathbf{W}_{\tau(v)}^{out} \in \mathbb{R}^{(d_K \times C)}$ where C is the number of classes and $\mathbf{h}_v^{(K)}\mathbf{W}_{\tau(v)}^{out}$ are the logits
  - Note: there is a separate matrix for each node type $\tau(v)$
- For heterogeneous graph classification:
  - Create a prediction head for each node type, $\tau(v)$, to predict class label (e.g., global mean pooling ⟶ linear layer)
  - Aggregate (e.g., mean) predictions across prediction heads

Edge-level prediction head

- Use pairs of node embeddings as input to a prediction function
- $\hat{y}_{uv} = g\left(\mathbf{h}_v^{(K)}, \mathbf{h}_u^{(K)}\right)$
- What are the options for $g\left(\mathbf{h}_v^{(K)}, \mathbf{h}_u^{(K)}\right)$?

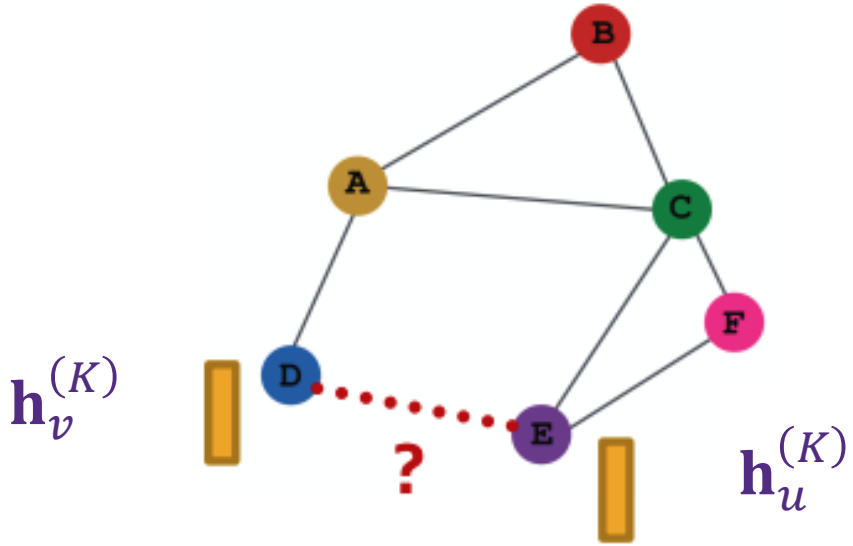


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Edge-level prediction head with concatenation

- One option is to concatenate the u and v node embeddings
- Prediction function, g, operates on the concatenated vector
- Example: g is a linear layer, $\hat{y}_{uv} = g\left(\left[\mathbf{h}_u^{(K)} \mid \mathbf{h}_v^{(K)}\right]\right) = W^g\left[\mathbf{h}_u^{(K)} \mid \mathbf{h}_v^{(K)}\right]$

- Here, $W^g \in \mathbb{R}^{C \times 2d_K} \rightarrow$ maps node embeddings from $\mathbb{R}^{2d_K}$ to $\mathbb{R}^C$

Account for the concatenation of embeddings

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Edge-level prediction head with dot product

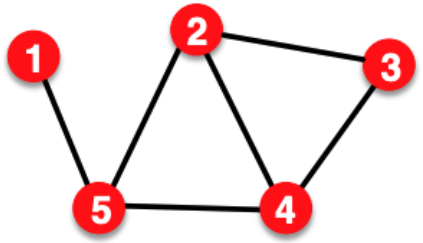
- The prediction head function is just the dot product of the node embeddings
  - $\hat{\boldsymbol{y}}_{uv} = \mathbf{h}_u^{(K)^T} \cdot \mathbf{h}_v^{(K)}$
  - This only applies to 1-way prediction (i.e., binary classification, single regression, link prediction)
- Applying to C-way prediction with a learnable generalized dot product
  - Learn c weight matrices

$$\hat{y}_{uv,1} = \mathbf{h}_u^{(K)^T} \mathbf{W}^1 \mathbf{h}_v^{(K)}$$
$$\vdots$$
$$\hat{y}_{uv,C} = \mathbf{h}_u^{(K)^T} \mathbf{W}^c \mathbf{h}_v^{(K)}$$
$$\hat{\boldsymbol{y}}_{uv} = \left[\hat{y}_{uv,1}, \cdots, \hat{y}_{uv,C}\right] \in \mathbb{R}^C$$

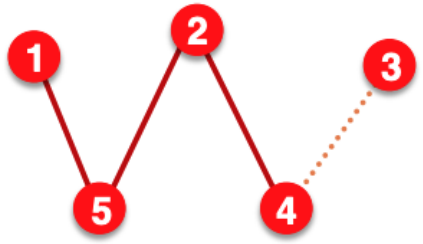
where $W^c \in \mathbb{R}^{d_K \times d_K}$

Setting up edge prediction

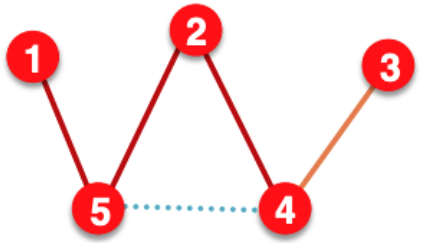
- Transductive link prediction split



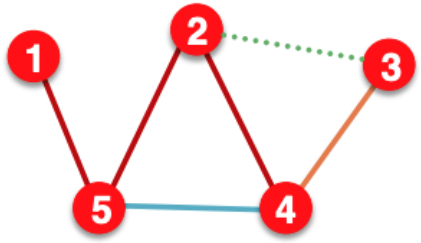
The original graph



(1) At training time:
Use **training message edges** to predict **training supervision edges**



(2) At validation time:
Use **training message edges & training supervision edges** to predict **validation edges**

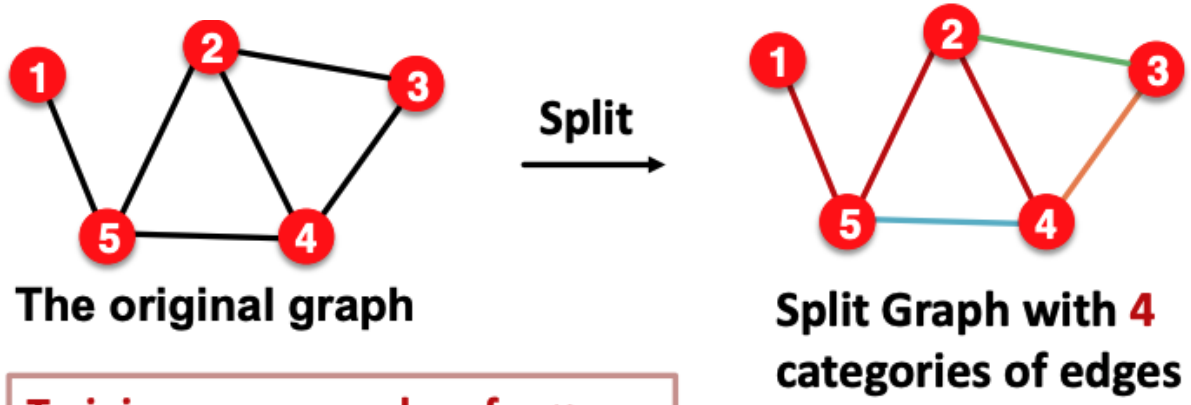


(3) At test time:
Use **training message edges & training supervision edges & validation edges** to predict **test edges**

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Setting up edge prediction

- Edge prediction split



Every edge has a relation type, independent of the four (4) categories of splits

For a heterogeneous graph, the homogeneous graphs formed by a given relation, has four (4) splits

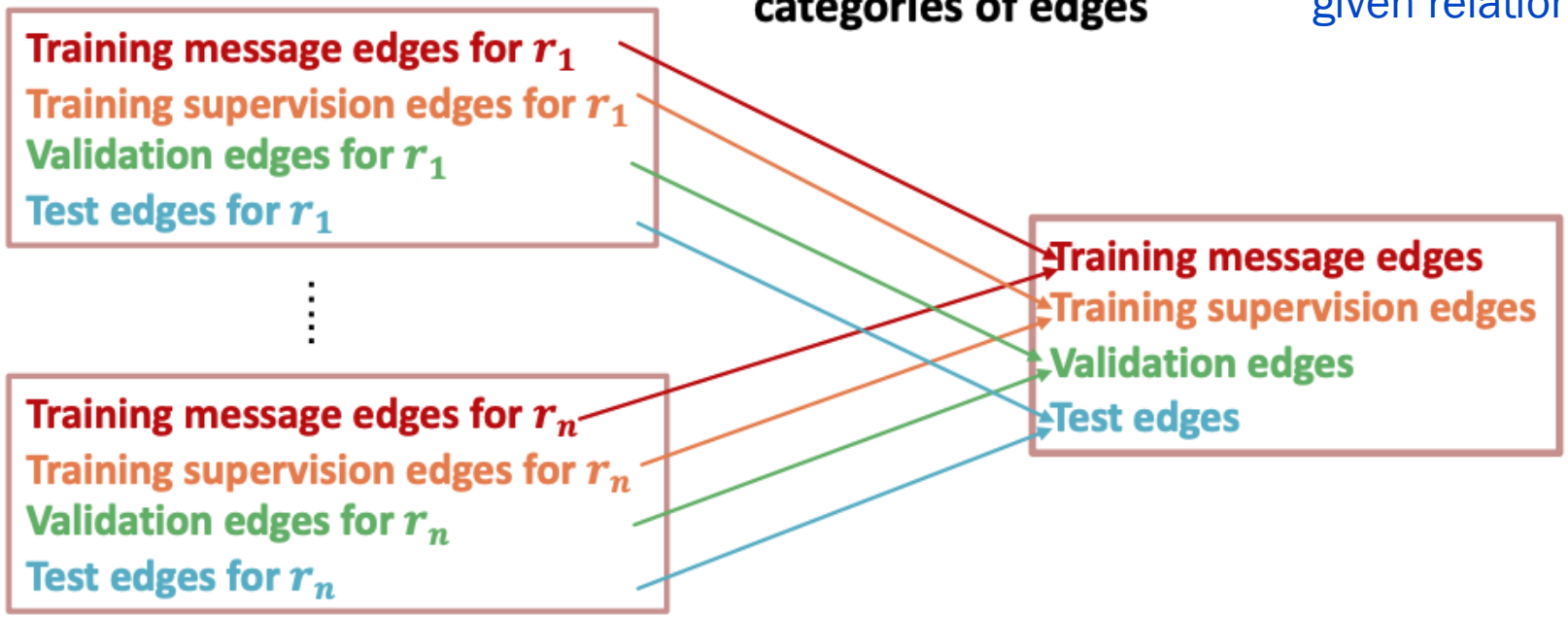


Image credit: Jure Leskovec, http://cs224w.stanford.edu

RGCN for edge prediction

- Assume *(E, r_3, A)* is a training supervision edge, and all other edges are training message edges
  - Other edges (not shown yet) are validation or test supervision edges
- Use the RGCN to predict probability for edge *(E, r_3, A)*
  - Take the final RGCN layer embeddings, $\mathbf{h}_E^{(K)}$ and $\mathbf{h}_A^{(K)}$, for *E* and *A*
  - Apply a relation specific prediction head g_r: $\mathbb{R}^{d_K \times d_K} \rightarrow \mathbb{R}$
    - Example: $g_{r_3} = \left(\mathbf{h}_E^{(K)}\right)^T \mathbf{W}_{r_3} \mathbf{h}_A^{(K)}$ where $\mathbf{W}_{r_3} \in \mathbb{R}^{d_K \times d_K}$

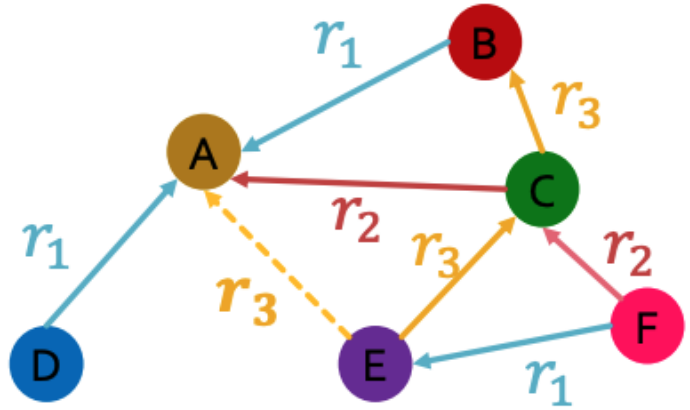


Image credit: Jure Leskovec, http://cs224w.stanford.edu

RGCN training for edge prediction

1. Use the RCGN to score the training supervision edge *(E, r_3, A)*
   - Message passing occurs only on the training message edges for this relation type
2. Create negative edge(s) by perturbing the supervision edge *(E, r_3, A)*, for example by changing the tail: *(E, r_3, B)*, *(E, r_3, F)*
   - Negative edges should not be in the actual graph (i.e., not a member of any of the 4 splits)

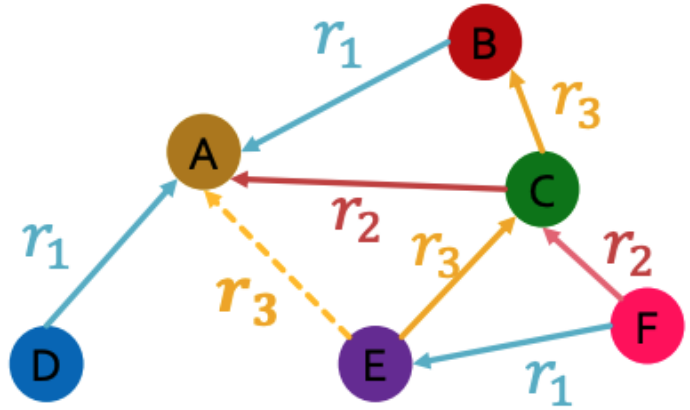


Image credit: Jure Leskovec, http://cs224w.stanford.edu

RGCN training for edge prediction

1. Use the RCGN to score the training supervision edge *(E, r_3, A)*
   - Message passing occurs only on the training message edges for this relation type
2. Create negative edge(s) by perturbing the supervision edge *(E, r_3, A)*, for example by changing the tail: *(E, r_3, B)*, *(E, r_3, F)*
   - Negative edges should not be in the actual graph (i.e., not a member of any of the 4 splits)
3. Score the negative edge with the RCGN
4. Optimize cross-entropy loss function

$$l = -log\sigma\left(g_{r_3}\left(\mathbf{h}_E^{(K)}, \mathbf{h}_A^{(K)}\right)\right) - \log\left(1 - \sigma\left(g_{r_3}\left(\mathbf{h}_E^{(K)}, \mathbf{h}_B^{(K)}\right)\right)\right)$$

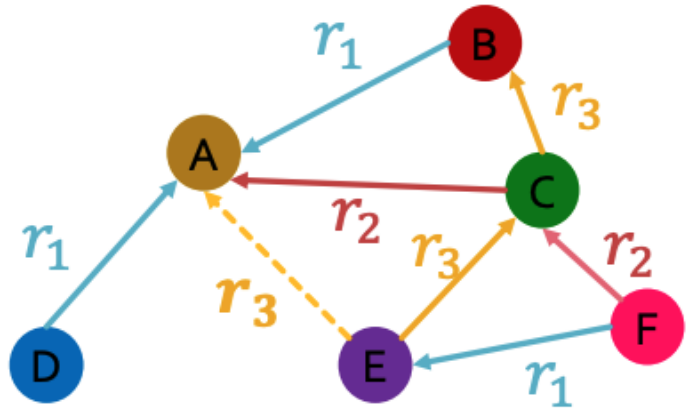


Image credit: Jure Leskovec, http://cs224w.stanford.edu

RGCN evaluation for edge prediction

- How should we evaluate model performance?
- We could use standard binary classification performance metrics, but these do not align with the typical use case
  - Most applications involve very large, sparse graphs
  - Nearly all “edge” predictions are negative
  - The typical requirement is ”precision”, more specifically, if the model indicates there are K predicted edges that are “missing” from the graph, how many of these are valid?
    - Intuition: we want the score for validation / test supervision edges to be greater than negative samples, without worrying about the normalized probability

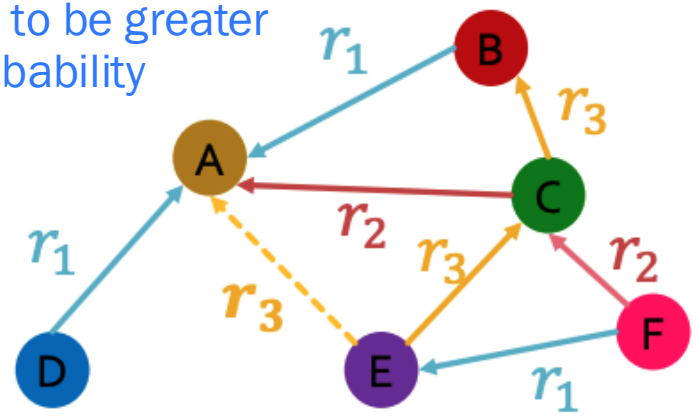


Image credit: Jure Leskovec, http://cs224w.stanford.edu

RGCN evaluation for edge prediction

- Consider evaluation during validation (same procedure for test)
- Assume *(E, r_3, D)* is a validation supervision edge
- Use the RCGN to score *(E, r_3, D)* and sampled negative edges, let's assume those are *(E, r_3, B), (E, r_3, F)*
- Rank the edges by their RCGN score
- Obtain the rank, R_K, of the *(E, r_3, D)*, where K is the number of edges considered
- Repeat this for all validation edges
  - Compute *Hits@k* – that is, how many of the validation supervision edges have a rank of $k < K$ or better? (Higher number of "hits" is better)
  - Reciprocal rank $\frac{1}{R_K}$ (Higher is better – indicates lower R_K)

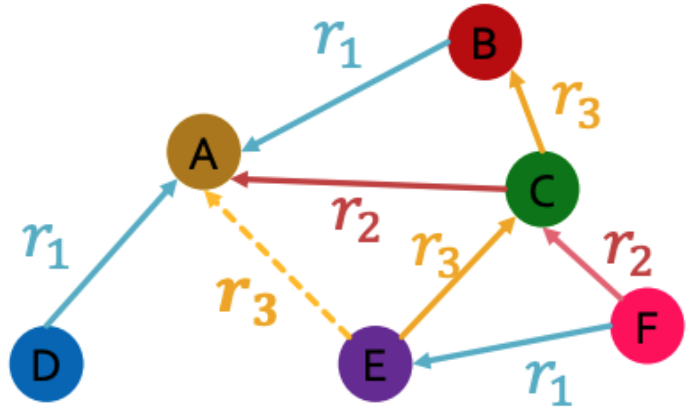


Image credit: Jure Leskovec, http://cs224w.stanford.edu

RGCN evaluation for edge prediction

- Consider evaluation during validation (same procedure for test)
- Assume *(E, r_3, D)* is a validation supervision edge
- Use the RCGN to score *(E, r_3, D)* and sampled negative edges, let's assume those are *(E, r_3, B), (E, r_3, F)*
  - Here $K = 3$ Let's consider Hits@2 (i.e, $k = 2$)
- Assume we get following ordering

1. g_{r_3}(E, r_3, D)
2. g_{r_3}(E, r_3, B)
3. g_{r_3}(E, r_3, F)

R_K is 1 *(E, r_3, D)*

- Increment hits@2 by 1
- Reciprocal rank is 1 for this sample

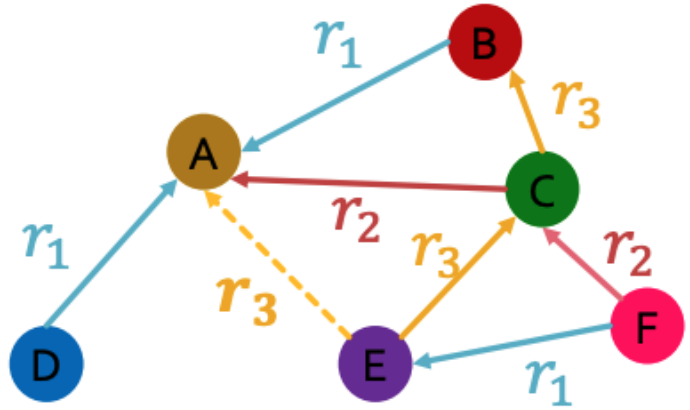


Image credit: Jure Leskovec, http://cs224w.stanford.edu

RGCN evaluation for edge prediction

- Consider evaluation during validation (same procedure for test)
- Assume *(E, r_3, D)* is a validation supervision edge
- Use the RCGN to score *(E, r_3, D)* and sampled negative edges, let's assume those are *(E, r_3, B), (E, r_3, F)*
  - Here $K = 3$ Let's consider Hits@2 (i.e, $k = 2$)
- Assume we get following ordering

1. g_{r_3}(E, r_3, B)
2. g_{r_3}(E, r_3, D)
3. g_{r_3}(E, r_3, F)

R_K is 2 (E, r_3, D)

- Increment hits@2 by 1
- Reciprocal rank is 1/2 for this sample

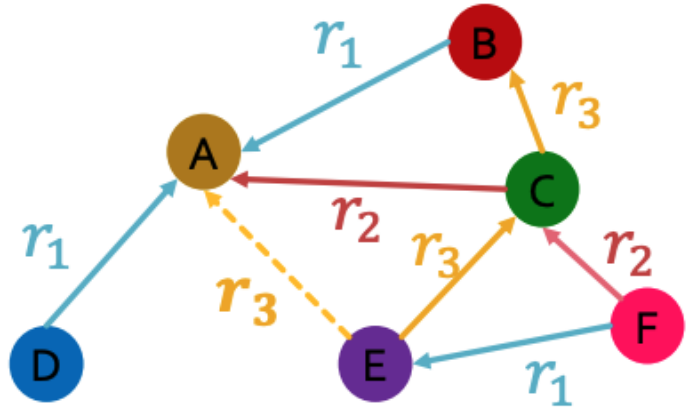


Image credit: Jure Leskovec, http://cs224w.stanford.edu

RGCN evaluation for edge prediction

- Consider evaluation during validation (same procedure for test)
- Assume *(E, r_3, D)* is a validation supervision edge
- Use the RCGN to score *(E, r_3, D)* and sampled negative edges, let's assume those are *(E, r_3, B), (E, r_3, F)*
  - Here $K = 3$ Let's consider Hits@2 (i.e, $k = 2$)
- Assume we get following ordering

1. g_{r_3}(E, r_3, B)
2. g_{r_3}(E, r_3, F)
3. g_{r_3}(E, r_3, D)

R_K is 3 (E, r_3, D)

- Increment hits@2 by 0 (no credit for this sample)
- Reciprocal rank is 1/3 for this sample

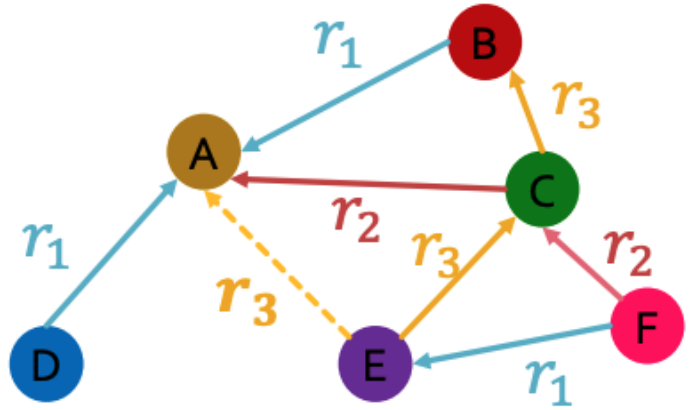
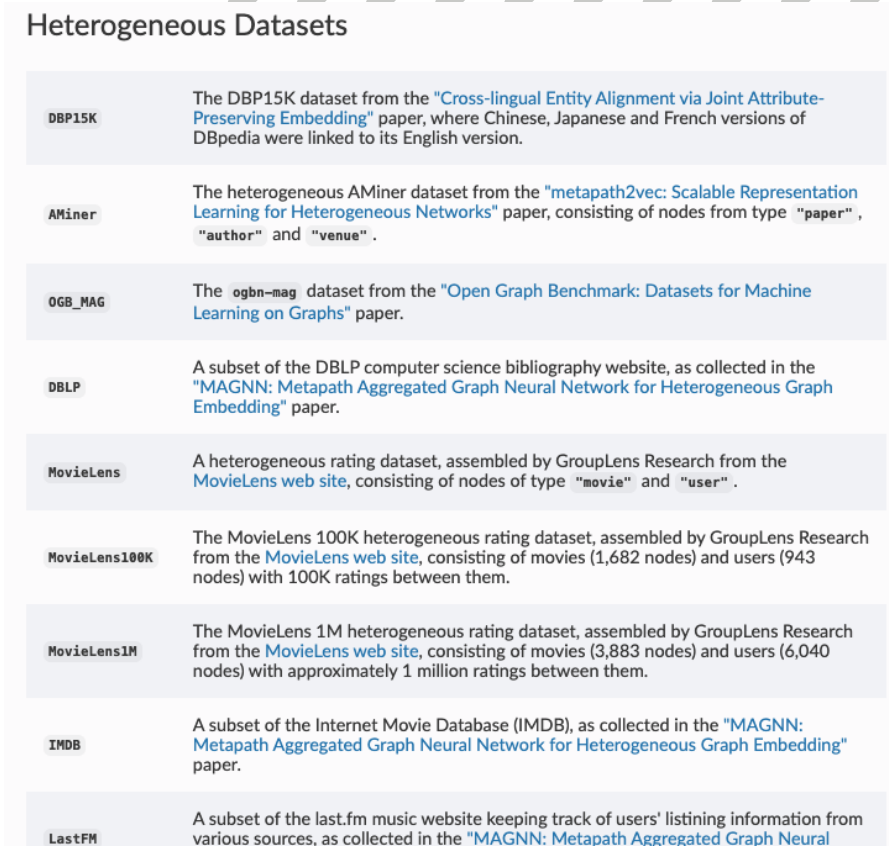


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Benchmarking heterogeneous graph methods

Numerous heterogeneous graph datasets in PyG



Heterogeneous Datasets

`DBP15K`	The DBP15K dataset from the "Cross-lingual Entity Alignment via Joint Attribute-Preserving Embedding" paper, where Chinese, Japanese and French versions of DBpedia were linked to its English version.
`AMiner`	The heterogeneous AMiner dataset from the "metapath2vec: Scalable Representation Learning for Heterogeneous Networks" paper, consisting of nodes from type `"paper"`, `"author"` and `"venue"`.
`OGB_MAG`	The `ogbn-mag` dataset from the "Open Graph Benchmark: Datasets for Machine Learning on Graphs" paper.
`DBLP`	A subset of the DBLP computer science bibliography website, as collected in the "MAGNN: Metapath Aggregated Graph Neural Network for Heterogeneous Graph Embedding" paper.
`MovieLens`	A heterogeneous rating dataset, assembled by GroupLens Research from the MovieLens web site, consisting of nodes of type `"movie"` and `"user"`.
`MovieLens100K`	The MovieLens 100K heterogeneous rating dataset, assembled by GroupLens Research from the MovieLens web site, consisting of movies (1,682 nodes) and users (943 nodes) with 100K ratings between them.
`MovieLens1M`	The MovieLens 1M heterogeneous rating dataset, assembled by GroupLens Research from the MovieLens web site, consisting of movies (3,883 nodes) and users (6,040 nodes) with approximately 1 million ratings between them.
`IMDB`	A subset of the Internet Movie Database (IMDB), as collected in the "MAGNN: Metapath Aggregated Graph Neural Network for Heterogeneous Graph Embedding" paper.
`LastFM`	A subset of the last.fm music website keeping track of users' listining information from various sources, as collected in the "MAGNN: Metapath Aggregated Graph Neural

Summary of RGCN

- RGCN extends GCN to heterogeneous graphs
- Can perform node / graph classification and link prediction
- Ideas can be extended to other RGNN models such as RGraphSAGE, RGIN, RGAT